



UNIVERSITY
OF COLOGNE

WORKSHOP 10

Advanced Analytics and Applications

Agenda

1

Lecture Recap

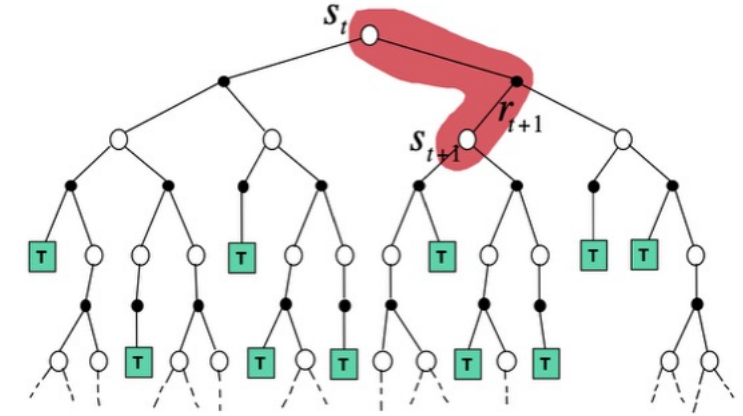
2

Exemplary Exam Questions

3

Coding Tasks

Temporal Difference Learners

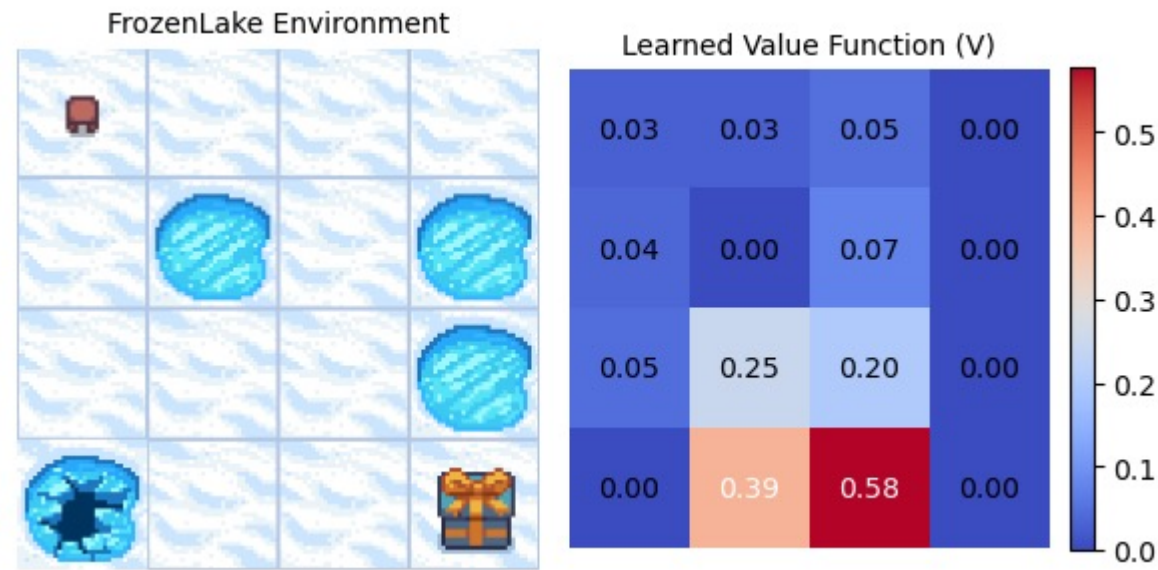


- In Monte Carlo we have to wait for an episode to finish to calculate the return.
- Temporal Difference (TD) methods **learn at every single step**, terminal or not, and they work in non-episodic environments as well. They **bootstrap** (like Dynamic Programming) but from **experience** (like Monte Carlo), combining the best of both worlds!
- Two core concepts:
 - The TD Target is an informed estimate of the state value $V(S_t)$
 - The TD Error is the difference between that and our old / current best estimate:

$$\delta_t = \underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD Target}} - V(S_t)$$

TD(0) for Evaluation

- The most trivial TD algorithm is TD(0) where we update our estimates using an update rule
$$V(S_t) \leftarrow V(S_t) + \alpha * \delta_t$$
- This allows us to evaluate a given policy and estimate state values under it.



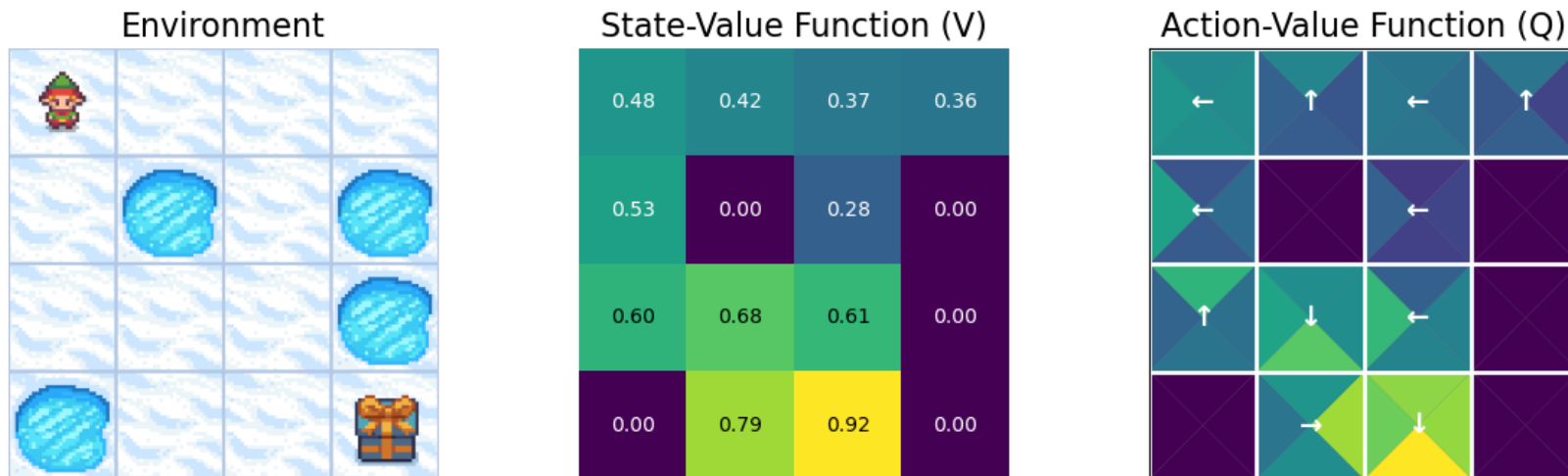
TD Control

- Now we can evaluate a policy (**prediction task**), but how do we find the optimal policy (**control task**)?
- This requires learning the value of an action Q , like in Monte Carlo!
- The two core concepts are adjusted, from $V(S_{t+1})$ to $Q(S_{t+1}, A_{t+1})$:
 - The TD Target is $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$
 - The TD Error is $\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$
- And we adjust the update rule: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha * \delta_t$

SARSA

- SARSA (short for the tuple it represents: **s**tate, **a**ction, **r**eward, **s**tate, **a**ction) is an *on-policy TD control* algorithm. **On-Policy** refers to the fact that the update rule uses the Q-value of the next action chosen by the policy it is currently following (including exploration), denoted by A' :

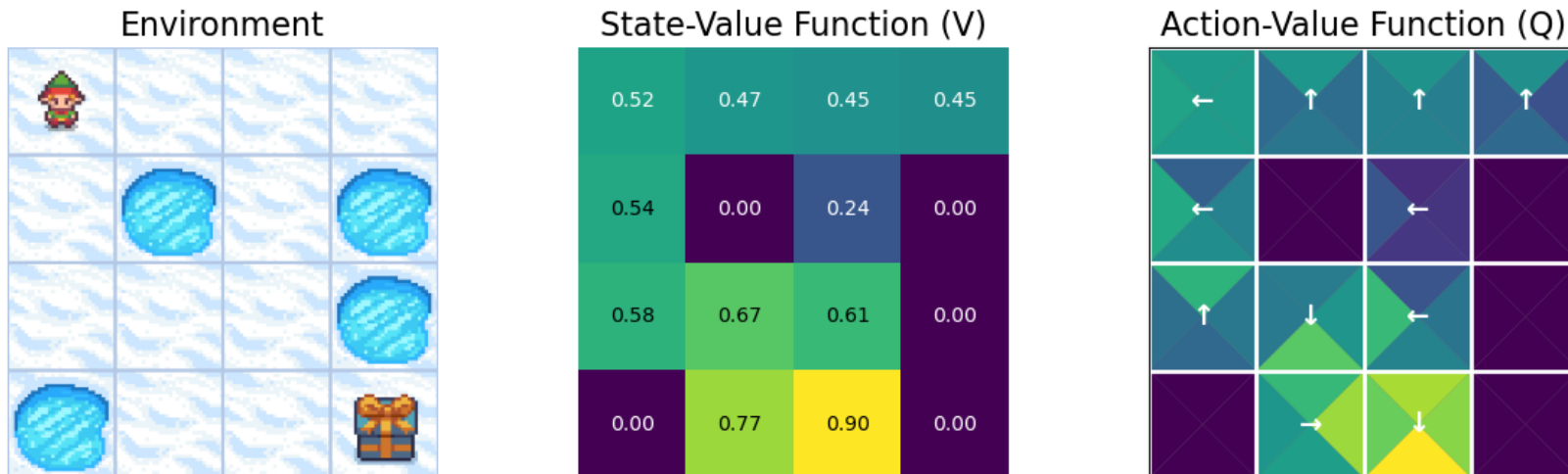
$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$$



Q-Learning

- Q-Learning is similar, but it is **off-policy** meaning it considers another Q-value in its update rule that is not a result of following its own policy but an optimal policy (the greedy one):

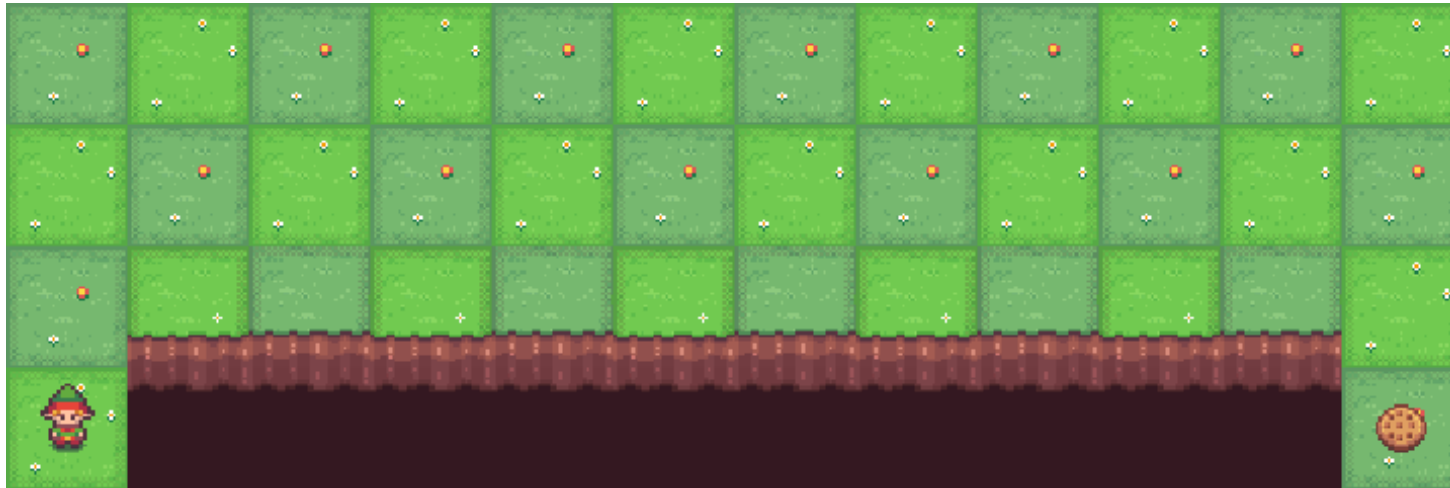
$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_{a'} Q(S', a') - Q(S, A)]$$



- Can you spot a difference to SARSA?

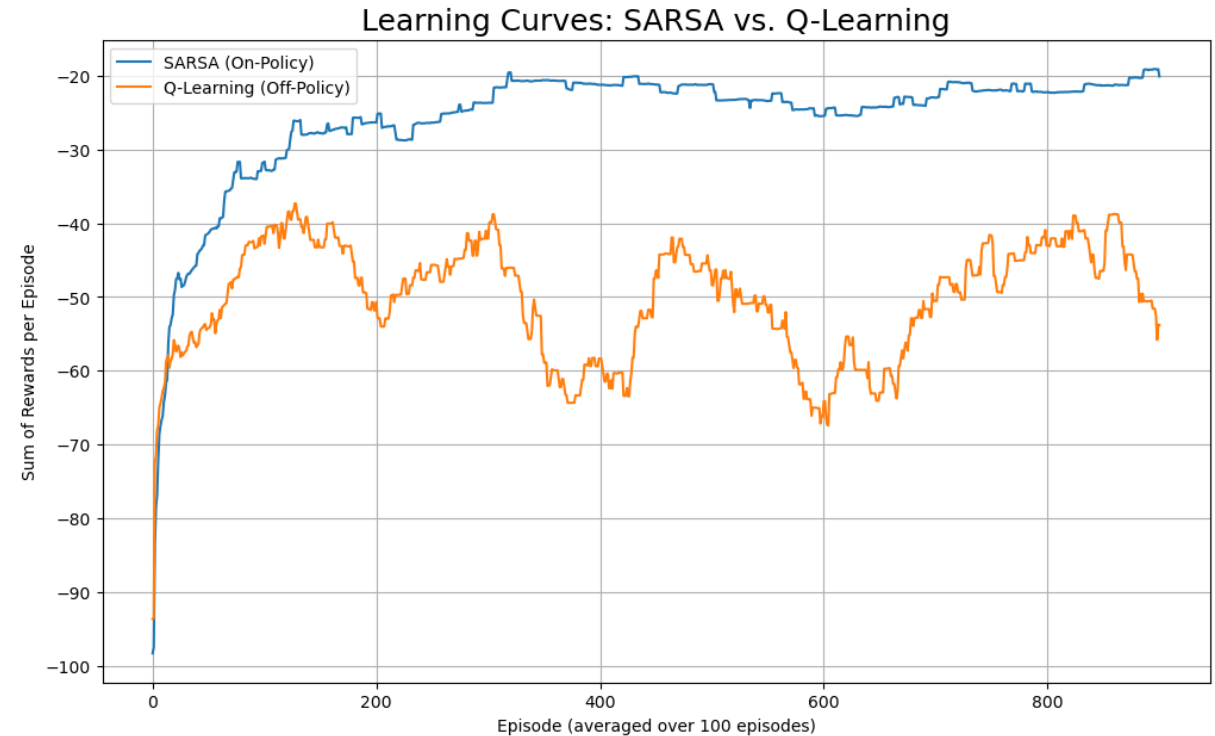
On-Policy vs. Off-Policy: Cliff Walk

- In the previous slides, Q-Learning was more optimistic or one could say more “greedy”.
- How does this affect the learner in an environment where the optimal route is the dangerous one?
- Example: *CliffWalk* environment. Large negative reward for falling, but also small negative reward for steps. The optimal route is the most daring one, right along the cliff!



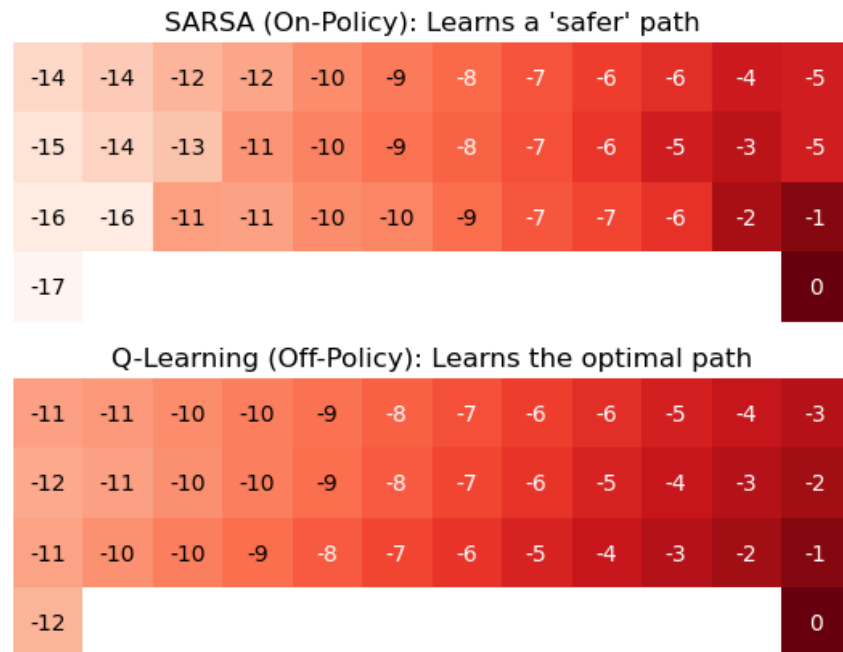
On-Policy vs. Off-Policy: Cliff Walk

- SARSA quickly learns to avoid the cliff.
- Q-Learning, while learning, explores, and falls into the cliff every once a while – because the policy for experiencing is not the same as the one that is learned!
 - *Always look at policy evaluation with Q-Learning, the evaluation performance during learning/training is deceptively bad!*



On-Policy vs. Off-Policy: Cliff Walk

Final State-Value (V) Functions: SARSA vs. Q-Learning



- The learned values for the states paint a clear picture:
 - SARSA learned that states near the cliff are dangerous (lower values compared to the rows above)
 - Q-Learning learned that states near the cliff are the optimal route to take because they minimize the penalty for longer routes.

Function Approximation: Q-Network

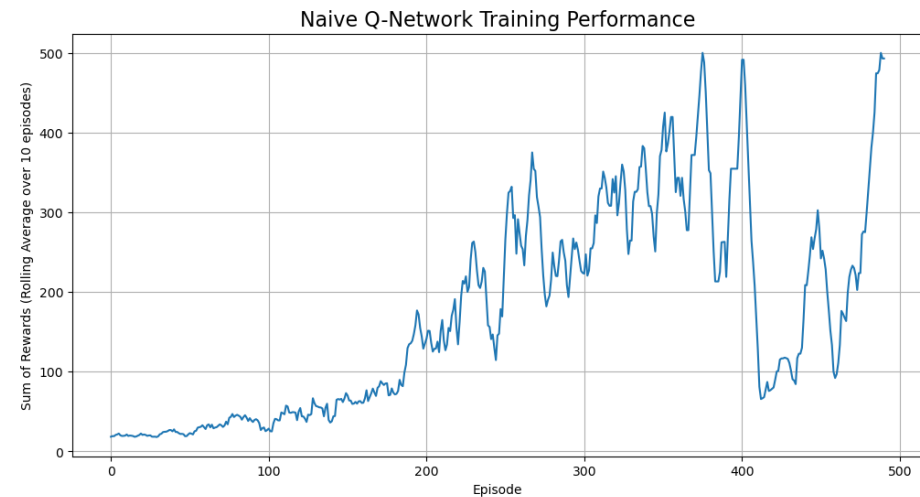
- Sometimes tables are not enough:
 - The **combinatorial number** of states and actions is simply **too large** (chess, Go, etc)
 - Or one of the two (state space or action space) is **continuous**
- We can replace the table of values with a **function** (or an approximation of it), parametrized by θ !
- We add a prediction problem, meaning we need a **loss function**. The good thing is, we already have a notion of how good our estimation is: The TD error! We can simply wrap that in error functions that we know from supervised learning, like the Mean Squared Error (MSE):

$$\mathcal{L}(\theta) = \left(\underbrace{R + \gamma \max_{a'} Q(S', a'; \theta)}_{\text{TD Target}} - \underbrace{Q(S, A; \theta)}_{\text{Prediction}} \right)^2$$

Function Approximation: Q-Network

- Simply plugging in a neural network for the Q-table does work in simple environments, but it suffers severe instability due to:
 - 1) Correlated Samples: The sequential nature of the MDP that generates observations, which we use to train the network is not *i.i.d.*, creating an inefficient training that learns certain trajectories and then has to throw that away and learn another certain trajectory.
 - 2) Moving Target: We use the same network, denoted by its weights θ for both the prediction and the target, making it hard for the network to converge!

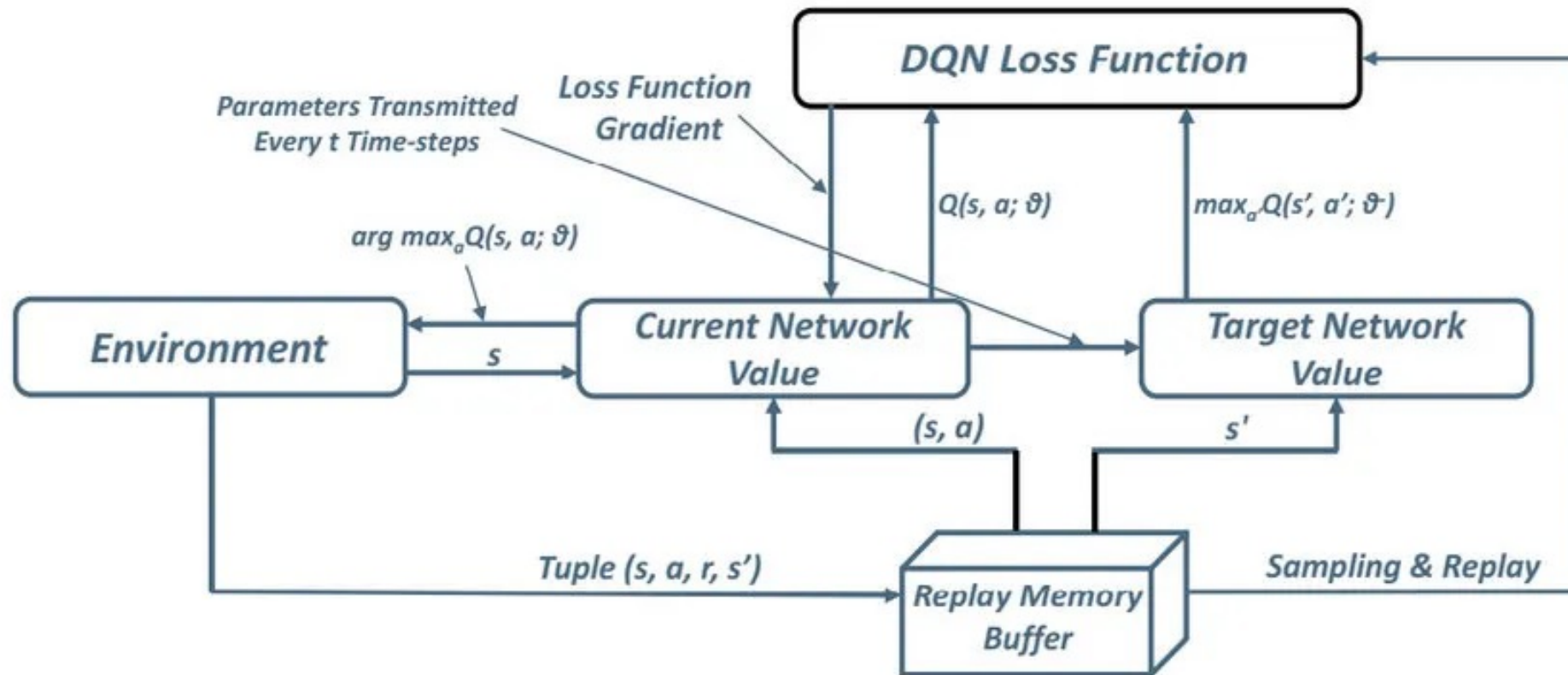
$$\mathcal{L}(\theta) = \left(\underbrace{R + \gamma \max_{a'} Q(S', a'; \theta)}_{\text{TD Target}} - \underbrace{Q(S, A; \theta)}_{\text{Prediction}} \right)^2$$



- Deep Q Networks add two stabilizing features:
 - **Experience Replay** to address the correlated samples, using an archive of experiences that we sample batches from. essentially making the training of the NN a supervised learning problem (just that, not the full learning of course).
 - **Target Network** to address the moving target. We simply create a copy of the Q-Network and use that for the TD target calculation. Periodically we update it from the main network by transferring the weights. This removes the reliance on the same network:

$$\mathcal{L}(\theta) = \left(R + \gamma \max_{a'} Q(S', a'; \hat{\theta}) - Q(S, A; \theta) \right)^2$$

DQN



Policy Gradient Methods

- Up until now, the goal was always to calculate/estimate **the value of taking a particular action**. This we could then exploit with a **deterministic policy**: $\operatorname{argmax}_a Q(s, a)$
- But what if actions are not well defined in discrete space? Especially in real-world applications!
 - Apply a certain amount of torque
 - Rotate the roboter's arm by a certain angle
 - ...
- And what if the optimal policy is not deterministic but **stochastic** (think of game-theoretic scenarios)
- This is where **policy gradient** methods come in:

We define the policy as a probabilistic function of taking action a in state s : $\pi(a|s)$

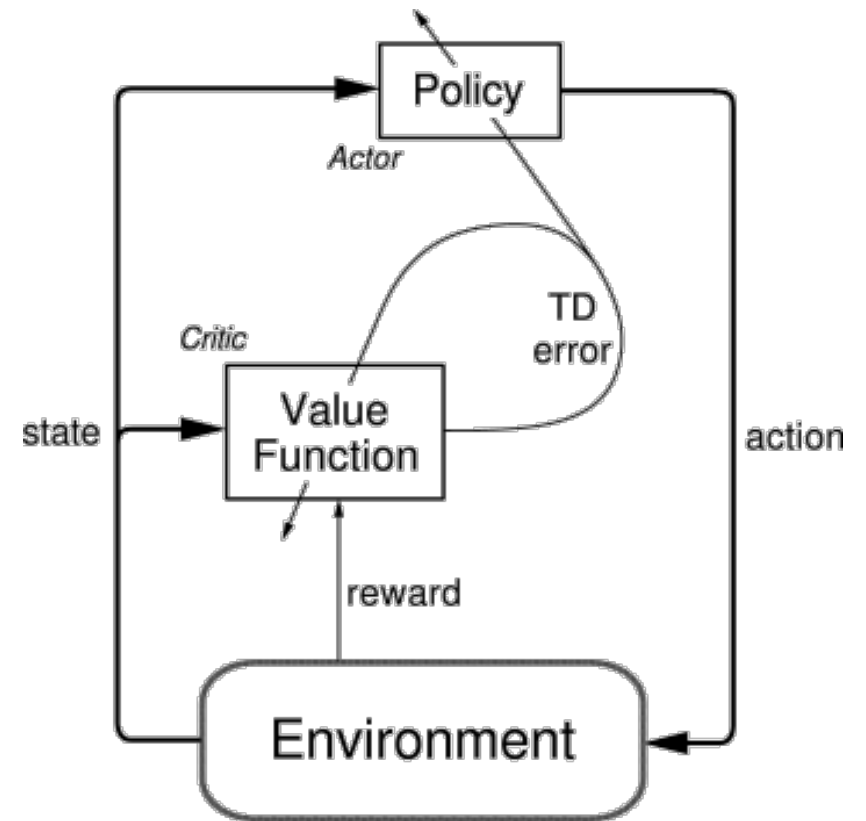
- Learning goal changes from minimizing value estimation error to **maximizing expected reward with π**

REINFORCE

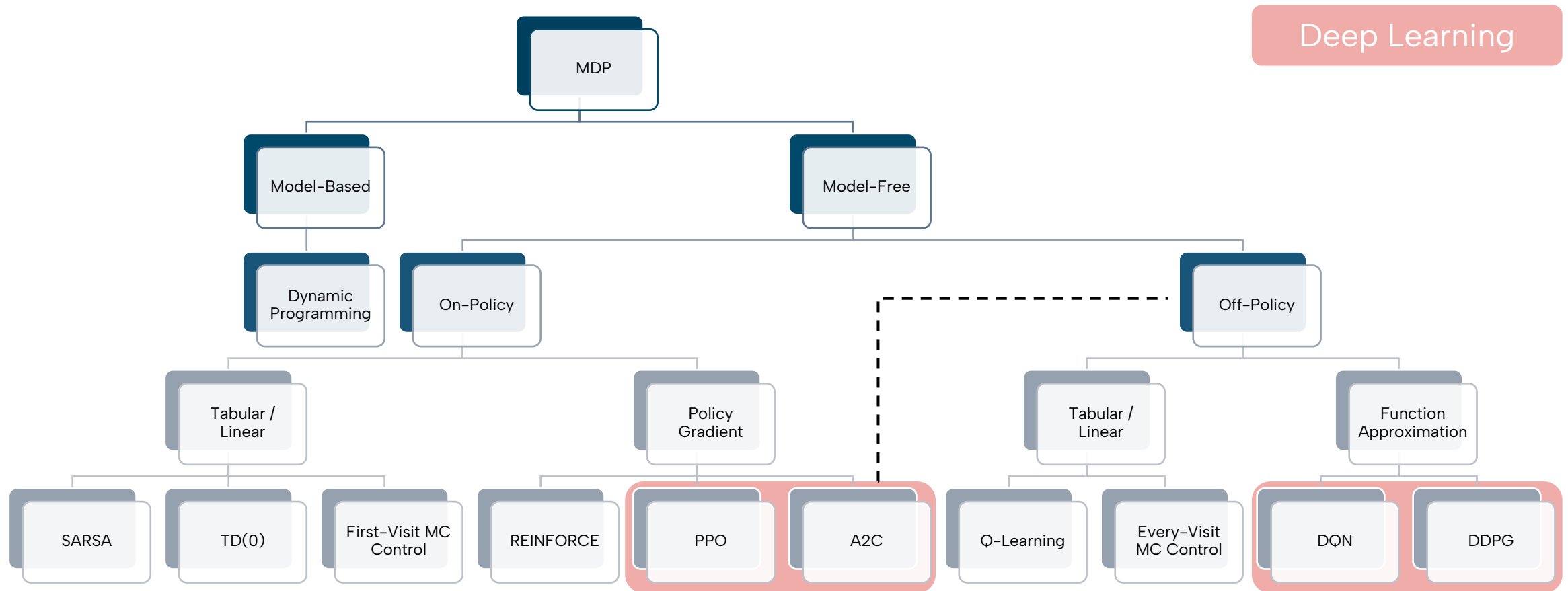
- The most simple policy gradient algorithm is an extension of Monte Carlo:
 - Experience one full episode, following the current policy π_θ
 - For every action at time t the agent took during that episode:
 - If the total discounted reward G_t was high, slightly increase $\pi(a|s)$
 - If the total discounted reward G_t was low, slightly decrease $\pi(a|s)$
- One drawback of REINFORCE is that every action's update is based on the total future return of the episode. The same starting state can lead to vastly different trajectories and vastly different returns! This means the return has a **high variance** for the same state across different episodes.
- One way to deal with that is using a **baseline** to subtract from the return to consider the change from the expectation. It can for example be the *value of a state* $V_\pi(s)$

Actor-Critic

- We can extend the baseline idea and combine it with TD updates.
- This means we don't need to wait for full episodes for the return to be available!
- We have an **actor** (policy network) that is responsible for deciding what to do (follow π) *[Usually predicting the parametrization of a distribution (like Gaussian), for every dimension of the action there are two output nodes, one for μ and one for σ]*
- We have a **critic** (value network) that learns state-value function to evaluate how good the actor's actions are.
- This means, the actor can directly update its policy by using the immediate (TD) feedback of the critic, instead of waiting for the full episode to finish, using $\delta \leftarrow r_n + \gamma V_w(s_{n+1}) - V_w(s_n)$
- Again, we can use an **advantage term** instead of the TD update to make it more robust, leading to **A2C** (Advantage Actor Critic)



Solving Decision Problems (Incomplete Overview)



Agenda

1

Lecture Recap

2

Exemplary Exam Questions

3

Coding Tasks

Temporal Difference

Question:

- a) Compare and contrast **SARSA** and **Q-Learning** based on their policy (on-policy vs. off-policy) and their update targets.
- b) Provide a concise scenario where one algorithm might be preferred over the other.

Temporal Difference

Question:

- a) Compare and contrast **SARSA** and **Q-Learning** based on their policy (on-policy vs. off-policy) and their update targets.

Answer:

- SARSA is on-policy. It learns the value of the policy it is currently following. The action A_{t+1} used in the update is the one *actually taken* by the agent's current policy in state s_{t+1} . The update target is
- Q-Learning is off-policy. It learns the value of the *optimal* policy, regardless of the policy being used to generate the experience (the behavior policy). The action a' used in the update is the *greedy action* with respect to the current Q-values in state s_{t+1} , not necessarily the action actually taken. The update target is

$$R_{t+1} + \gamma Q(S_{t+1}, \max_a Q(S_{t+1}, a))$$

Temporal Difference

Question:

b) Provide a concise scenario where one algorithm might be preferred over the other.

Answer:

- **SARSA** might be preferred in scenarios where **safe exploration** is critical. For example, training a robot where taking a truly greedy action (as Q-Learning would evaluate) could lead to dangerous or costly collisions. SARSA's updates are based on the actual actions taken, making it "safer" in environments where the optimal path might involve navigating near hazards, as it accounts for the risk of the next action.
- **Q-Learning** is generally preferred when the goal is to find the **optimal policy** and the agent can learn from diverse experiences, including those generated by a highly exploratory behavior policy. For example, in a game where the agent can freely explore and reset at no real cost, Q-Learning can be more sample-efficient as it can learn about the optimal path even if the current exploration policy is far from optimal.

Deep Q Networks

Question: A research team is developing an RL agent to control the **throttle percentage** (a value between 0.0 and 1.0) of an autonomous vehicle, aiming to maintain a target speed. The environment provides a continuous state representation (e.g., current speed, distance to target).

- a) Explain why a standard Deep Q-Network (DQN) architecture would be impractical or ineffective for this task, despite its success in real-world environments.
- b) What class of model-free RL algorithms would be more suitable for directly handling such a continuous action space?

Deep Q Networks

Question: A research team is developing an RL agent to control the **throttle percentage** (a value between 0.0 and 1.0) of an autonomous vehicle, aiming to maintain a target speed. The environment provides a continuous state representation (e.g., current speed, distance to target).

- a) Explain why a standard Deep Q–Network (DQN) architecture would be impractical or ineffective for this task, despite its success in real–world environments.

Answer: DQN is designed for discrete actions, even discretizing the action space would produce prohibitively many actions or severely limit the capability of the agent. The specific problem is that an *argmax* over an infinite set of possibilities is not well–defined, which DQN needs: $a^* = \operatorname{argmax}_a Q(s, a)$

Deep Q Networks

Question: A research team is developing an RL agent to control the **throttle percentage** (a value between 0.0 and 1.0) of an autonomous vehicle, aiming to maintain a target speed. The environment provides a continuous state representation (e.g., current speed, distance to target).

b) What class of model-free RL algorithms would be more suitable for directly handling such a continuous action space?

Answer: The class of model-free RL algorithms more suitable for directly handling continuous action spaces are **Policy Gradient methods** (e.g., REINFORCE, DDPG, PPO, A2C). These methods directly learn a parameterized policy $\pi_{\theta}(a | s)$ which can output a continuous action or parameters for a continuous distribution of actions, eliminating the need for an *argmax* over an infinite set.

Agenda

1

Lecture Recap

2

Exemplary Exam Questions

3

Coding Tasks

Contact



For general questions and enquiries on **research, teaching, job openings** and **new projects** refer to our website at www.is3.uni-koeln.de



For specific **enquiries regarding this course** contact us by sending an email to the **IS3 teaching address** at is3-teaching@wiso.uni-koeln.de

To help us process your request efficiently, please use the following subject line format:

[AAA] <Your request subject>